Syracuse University
School of Information Studies

Week 8
Introduction to Linear Modeling

Dr. Preeti Jagadev

Traditional Statistical Analysis

- **Goal:** You start with a theory or hypothesis (an idea you want to test) and use data to *confirm or reject it*.
- **Data quality:** The data is usually clean, reliable, and well-prepared.
- **Planning:** You decide in advance *what to measure* and *how* (for example, using surveys or experiments).
- **Data type:** Most of the data is numerical and follows standard patterns (like a normal bell curve).

Data Mining / Machine Learning

- **Goal:** You don't start with a theory; you explore the data to *discover new patterns or insights*.

- **Data quality:** The data might be messy, incomplete, or unorganized, and you often have to clean it a lot.

- **Planning:** There's no careful planning of what to measure; you just work with whatever data is available.

- **Data type:** Data can be mixed, numbers, text, categories, and doesn't follow any fixed statistical pattern.

- **Train, Validate, Test**

Machine Learning

Unsupervised Learning:

- **K-means clustering:** Groups similar items together.
- **Association rule mining:** Finds relationships (like “people who buy bread also buy butter”).
- Think of it like exploring data to *discover hidden structures*.

Supervised Learning (mainly used for classification and regression):

Example techniques:

- **Neural networks**
- **Support vector machines (SVM)**
- **Regression trees and boosted trees**
- **General additive models**

Reinforcement Learning

- The model **learns by trial and error**.
- It gets rewards for doing things right and penalties for doing things wrong.
- Common in robots, games, and simulations.

The Machine Learning Process OSEMN

- **Obtain:** Collect data
- **Scrub:** Get data into a useable format structure (Filter, extract attributes, Replace/handle missing, illegal, and anomalous values**).** Typically, **50% or more** of the work is in this phase.
- **Explore:** Explore patterns and trends (Detect outliers, visualize descriptive statistics, Feature selection).
- **Model:** Build predictive models (Supervised, Unsupervised).
- **Interpret:** Understand/explain the results

Two Types of Prediction Problems in Machine Learning

Classification Problem:

- **Goal:** Predict *which category* something belongs to.
- The answer is **a label or class** (not a number).
- **Examples:**
  - Will a patient be sick or healthy? (medical diagnosis)
  - Will an applicant get hired or not? (employment outcome)

Two Types of Prediction Problems in Machine Learning

Regression Problem:

- **Goal:** Predict a **number** (a continuous outcome).
- The answer is a **numeric value,** not a category.
- In regression, the predicted variable is also known as the: **Dependent Variable.**
- **Examples:**
  - How many sales will we make next month?
  - What is the customer satisfaction rating?

Linear Modeling

- Imagine you have a bunch of points on a graph: The **x-axis** is something you know or control (like hours studied).The **y-axis** is something you want to predict (like exam score).

- The **regression line** is a straight line that tries to go through the middle of all the points so that it best represents the relationship between x and y.

- The equation of a **regression line:**

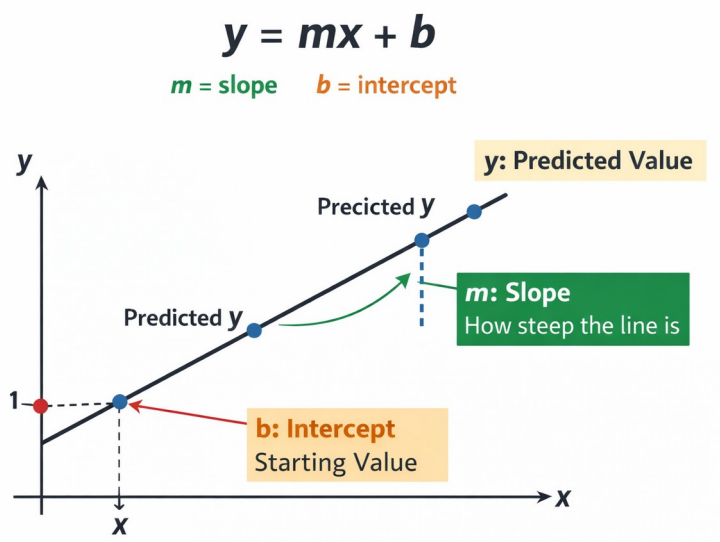
$$y=mx+b$$

y → what we want to predict (like score)

x → what we know or control (like hours studied)

m → slope (Tells how much y changes when x increases by 1)

b → intercept (Tells the value of y when x = 0)



Example:

x = hours studied

y = score

Intercept **b** = 10 → even if you don't study, you get 10 points

Slope **m** = 2 → each extra hour studied adds 2 points

So, if someone studies 3 hours:

They're predicted to score 16 points.

Hours Studied (x)	Calculation (y = 2x + 10)	Predicted Score (y)
0	2×0 + 10	10
1	2×1 + 10	12
2	2×2 + 10	14
3	2×3 + 10	16
4	2×4 + 10	18
5	2×5 + 10	20

Prediction Errors: The Least Squares Criterion

Prediction Line

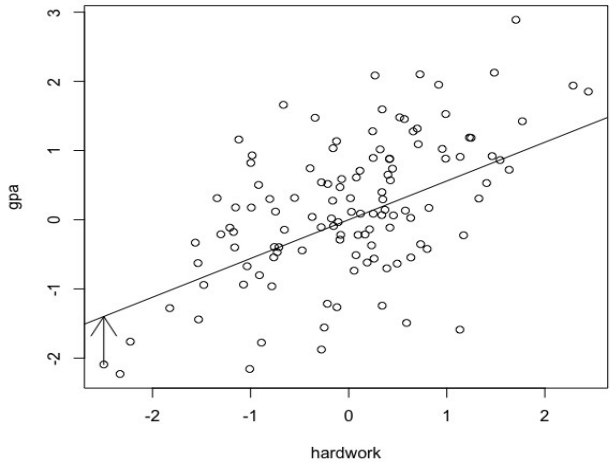
- Imagine the straight line on the graph is the **model's guess** for each student's GPA.
- This line comes from the equation

Actual Points

- The dots are the real GPAs students got.

Prediction Error (Residual)

- The **error** is how far the model's prediction (the line) is from the real value (the dot).
- Example:
  - **Dot above the line** → model guessed too low → **positive error**
  - **Dot below the line** → model guessed too high → **negative error**



The problem with errors:

- Your model predicts something, like a student's GPA.
- Some predictions are **too low** → positive errors
- Some predictions are **too high** → negative errors
- If you just **add them up**, the positive and negative errors can **cancel each other out**.
- **Example**: +3 and -3 → sum = 0
- But clearly the model **made mistakes**, we just can't see it by adding.

The solution: Sum of Squared Errors (SSE)

- Take **each error** and **square it.**
- Squaring makes **all errors positive**, so nothing cancels out.
- Add up all these squared errors → this is called **Sum of Squared Errors (SSE)**.

The best line

- The regression line tries to be the **closest to all points**.
- The line is **best** when the **SSE is as small as possible**.

Coding

```r
library(ggplot2)

install.packages("MASS")

library(MASS)

boston_df <- Boston

str(boston_df)
```

- **library(ggplot2):** This package allows you to create graphs and visualizations in R.
- **install.packages("MASS"):**
  - **MASS** stands for *Modern Applied Statistics with S.*
  - It is a package that comes with **many statistical tools and example datasets.**
- **boston_df <- Boston:**
  - The MASS package has a built-in dataset called **Boston.**
  - This line copies that dataset into a new variable called **boston_df.**
  - The **Boston dataset** is a housing dataset.
  - It contains information about houses and neighborhoods in Boston.
  - It has: 506 rows → **506 different neighborhoods**, 14 columns → **14 different features** (characteristics).

```r
sum(is.na(boston_df$rm))

## [1] 0
sum(is.na(boston_df$dis))

## [1] 0
sum(is.na(boston_df$medv))

## [1] 0
```

sum(is.na(boston_df$rm))

- **boston_df$rm:** This selects the **rm column** (average number of rooms).
- **is.na(...):** This checks: "Is any value missing?"
- **sum(...):** This counts how many missing values there are.
- **0:** This is the output. It means **0 missing values**.

sum(is.na(boston_df$dis))

- **boston_df$dis:** This selects the **dis column** (distance to job centers).
- **is.na(...):** This checks: "Is any value missing?"
- **sum(...):** This counts how many missing values there are.
- **0:** This is the output. It means **0 missing values**.

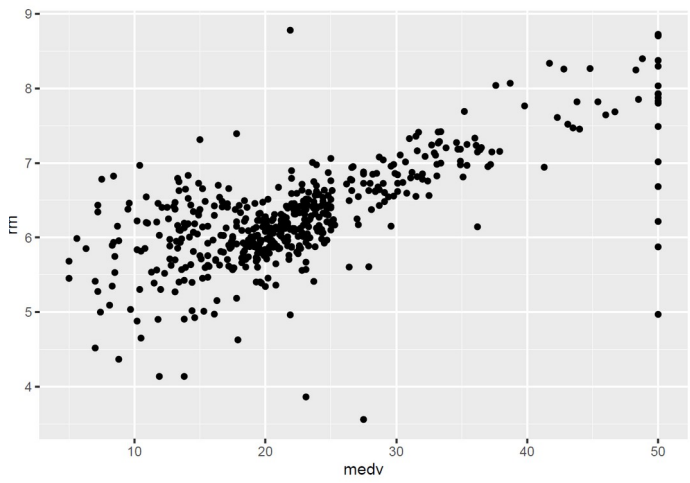
sum(is.na(boston_df$medv))

- **boston_df$dis:** This selects the **medv column** (median house value).
- **is.na(...):** This checks: "Is any value missing?"
- **sum(...):** This counts how many missing values there are.
- **0:** This is the output. It means **0 missing values**.

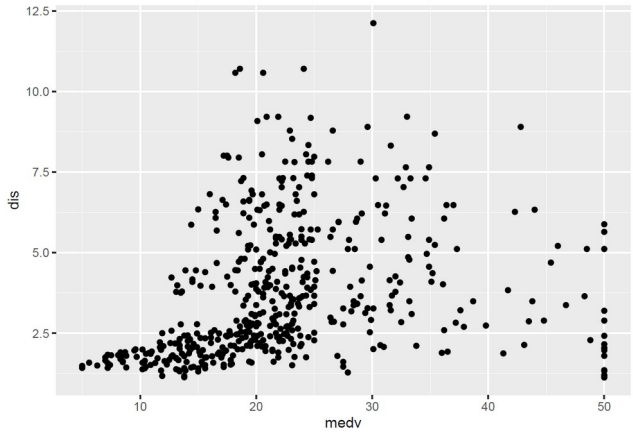
R doesn't like it when numeric columns in your data frame have NAs because it can't perform math functions on them properly. That's why it's important to detect and transform NAs into something meaningful.

- For our analysis, we will focus on **medv** as the outcome variable and explore how it relates to **rm** and **dis** as predictor variables.

```
ggplot(data=boston_df) + aes(x=medv, y=rm) + geom_point()
```




```
ggplot(data=boston_df) + aes(x=medv, y=dis) + geom_point()
```

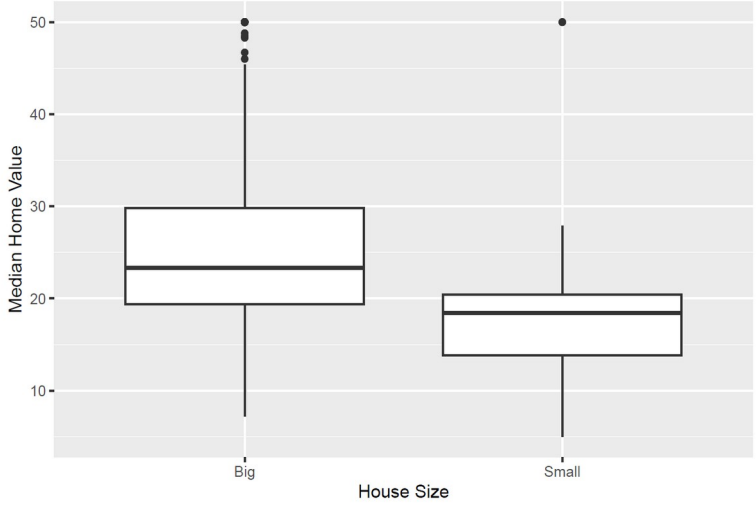


```r
boston_df$rm_binary <- ifelse(boston_df$rm >= 6, "Big", "Small")

ggplot(data = boston_df, aes(x = rm_binary, y = medv)) +
  geom_boxplot() +
  labs(x = "House Size", y = "Median Home Value", title = "Boxplot of Median Home Value by House Size")
```

boston_df$rm_binary <- ifelse(boston_df$rm >= 6, "Big", "Small"):

- **boston_df$rm_binary**→ Name of the new column we are creating.
- **boston_df$rm >= 6:** Check if the average number of rooms is 6 or more.
- **"Big“:** If the house has 6 or more rooms, label it Big.
- **"Small“:** If the house has less than 6 rooms, label it Small.



ggplot(data = boston_df, aes(x = rm_binary, y = medv)):

- **x = rm_binary**: Put house size (Big/Small) on the x-axis.
- **y = medv:** Put median home value on the y-axis.

geom_boxplot()

- This tells R to draw a boxplot.

labs(x = "House Size", y = "Median Home Value", title = "Boxplot of Median Home Value by House Size"):

- This line adds labels to the graph.

Let's create a **linear regression model** where we try to predict changes in **medv** based on **rm**:

```r
lmOut <- lm(medv ~ rm, data=boston_df)
```

```r
summary(lmOut)
```

```
##
## Call:
## lm(formula = medv ~ rm, data = boston_df)
##
## Residuals:
##     Min      1Q  Median      3Q     Max
## -23.346  -2.547   0.090   2.986  39.433
##
## Coefficients:
##             Estimate Std. Error t value Pr(>|t|)
## (Intercept)  -34.671      2.650  -13.08   <2e-16 ***
## rm             9.102      0.419   21.72   <2e-16 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 6.616 on 504 degrees of freedom
## Multiple R-squared:  0.4835, Adjusted R-squared:  0.4825
## F-statistic: 471.8 on 1 and 504 DF,  p-value: < 2.2e-16
```

lmOut <- lm(medv ~ rm, data=boston_df)

- **lm():** This is a function in R that fits a linear model.
- **medv ~ rm:**
  - **medv** (the median value of houses) is the dependent variable (what we are trying to predict).
  - **rm (**average number of rooms per house**)** is the independent variable (what affects medv).
  - **~** means "**is modeled as a function of**".
- **data=boston_df:** This tells R which dataset to use.

Syracuse University
School of Information Studies

The End